

Lecture 7: Matrix decompositions

Let's go back to the idea of change of basis. Recall that a linear transformation $T: \mathbb{R}^n \rightarrow \mathbb{R}^m$ has its corresponding matrix form:

$$[T]_{\beta} = \begin{pmatrix} T(b_1) & T(b_2) & \dots & T(b_n) \end{pmatrix} \in \mathbb{R}^{m \times n}$$

where $\beta = \{b_1, \dots, b_n\}$ is a basis for $\mathbb{R}^n$. Thus, the matrix form of a linear transformation is not unique, since a different choice of basis would give a different matrix.

In many cases, we are given already a matrix $[T]_{\beta}$ but wish to "view" this transformation with a different basis $\beta' = \{b'_1, \dots, b'_n\}$.

Then:

Given $[T]_{\beta}$, can we find $[T]_{\beta'}$ without directly computing

$T(b'_i)$? Answer: Yes. For simplicity: $\mathbb{R}^n = \mathbb{R}^m$.

First, notice that if $x \in \mathbb{R}^n$ is written in coordinates as $[x]_{\beta}$, then

$$S[x]_{\beta'} = [x]_{\beta} \rightarrow \text{This tells me that: if } [x]_{\beta} = (x_1, \dots, x_n), \text{ then}$$

$$\text{where } S = \begin{pmatrix} [b'_1]_{\beta} & [b'_2]_{\beta} & \dots & [b'_n]_{\beta} \end{pmatrix}.$$

$$x_i = \sum_{j=1}^n S_{ij} x'_j; \quad S_{ij} = \begin{matrix} i\text{-th coordinate} \\ \text{of } [b'_j]_{\beta} \end{matrix}$$

$$\text{where } [x]_{\beta'} = (x'_1, \dots, x'_n).$$

It is easy to see that $[x]_{\beta'} = S^{-1}[x]_{\beta}$ (i.e. S^{-1} allows us to change the basis back).

Then, if we had a mapping F , written in matrix form w.r.t. β :

$$[F]_{\beta}$$

To find $[F]_{\beta'}$, i.e., in the new basis, we perform:

S^{-1} puts everything back together in β' -coordinates.

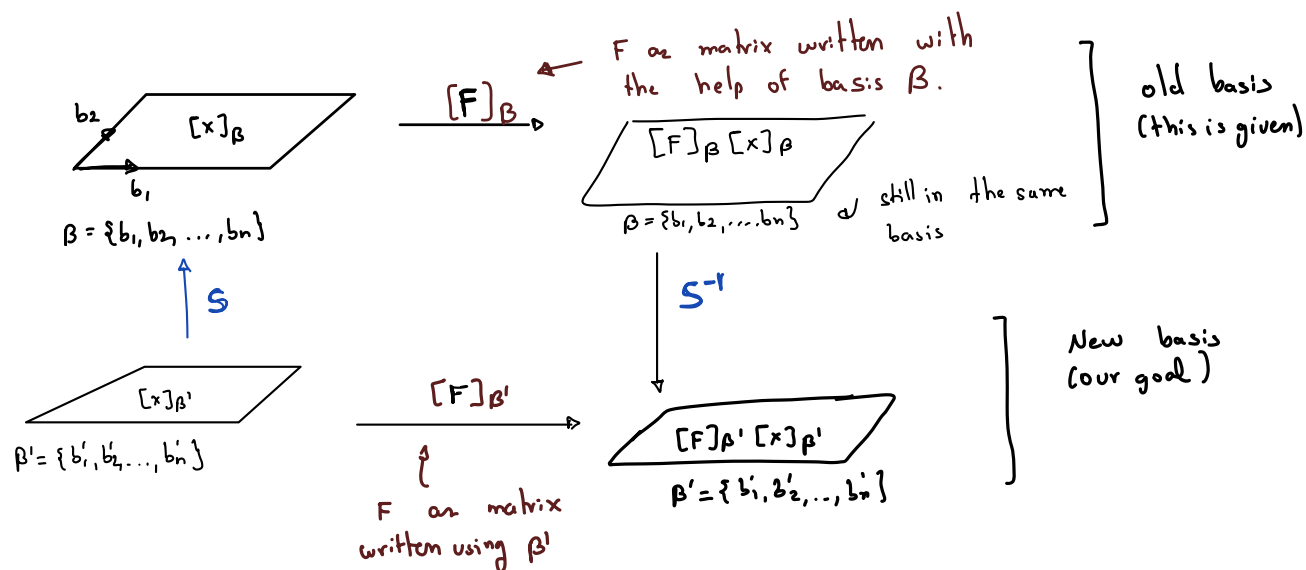
$$[F]_{\beta'} = S^{-1} [F]_{\beta} S$$

S changes the " β' -coordinates" of something to the β -coordinates.

It lives in β' -world, so it needs as input something written w.r.t. β' .

$[F]_{\beta}$ lives in β -world, so it needs as input something written w.r.t. β . Its output is still in β .

Visually:



Obviously, $[F]_B$ and $[F]_{B'}$ still describe the same transformation, only written in a different basis. Hence we say $[F]_B$ and $[F]_{B'}$ are "similar".

This motivates the following definition.

► We say that two matrices are **similar** if there exists a matrix S invertible s.t.

$$B = S^{-1} A S$$

Why is this useful? Because now we can study a (in practice) more simple matrix B compared to A . For example, if $B = D$ some diagonal matrix $\Rightarrow$ then we say A is **diagonalizable** and

$$A = S D S^{-1}$$

For example, If A is symmetric, we know that its eigenvectors form an orthonormal basis for $\mathbb{R}^n$. Write these vectors as columns of Q :

$$Q = \begin{pmatrix} v_1 & \dots & v_n \end{pmatrix}$$

we can show the following **eigenvalue decomposition**:

$$A = Q D Q^{-1}, \text{ where } D = \text{diag}(\lambda_1, \dots, \lambda_n)$$

where $\lambda_1, \dots, \lambda_n$ are the (real) eigenvalues of A .

Notice that, since each column vector in Q is orthonormal:

$$v_i \cdot v_j = \delta_{ij} = \begin{cases} 1 & \text{if } i=j \\ 0 & \text{if } i \neq j \end{cases}$$

$$\begin{aligned} &\Downarrow \\ Q^T Q &= \begin{pmatrix} v_1^T v_1 & v_1^T v_2 & \dots & v_1^T v_n \\ v_2^T v_1 & v_2^T v_2 & \dots & v_2^T v_n \\ \vdots & \vdots & \ddots & \vdots \\ v_n^T v_1 & v_n^T v_2 & \dots & v_n^T v_n \end{pmatrix} = \begin{pmatrix} 1 & & & \\ & 1 & & \\ & & \ddots & \\ & & & 1 \end{pmatrix} \\ &= I. \end{aligned}$$

That is: $Q^{-1} = Q^T$.

Then, in this new eigenbasis, the transformation A acts as a multiplication, "scaling" the coordinates of all vectors (written in this eigenbasis):

$$Ax = Q D Q^{-1} x$$

Annotations:

- x written with standard basis in $\mathbb{R}^n$ (points to x)
- transforms $Q^{-1}x$ back to standard coordinates (points to $Q^{-1}x$)
- transforms x -coordinates to our eigenbasis coordinates (points to $Q^{-1}x$)
- multiplies coordinate i by λ_i (points to D)

Another advantage of "diagonalization": faster computations

• Exponential of A : $e^A := Q \operatorname{diag}(e^{\lambda_1}, e^{\lambda_2}, \dots, e^{\lambda_n}) Q^{-1}$

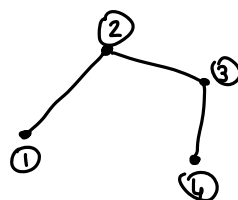
↑ actually $e^A = \sum_{i=0}^{\infty} \frac{A^i}{i!}$ and diagonalization helps show this nice formula above.

• Powers of A : $A^m = (Q^T D Q)^m$

$$\begin{aligned} &= \underbrace{(Q^T D Q)}_I \cdot \underbrace{(Q^T D Q)}_I \cdot \dots \cdot \underbrace{(Q^T D Q)}_I \underbrace{(Q^T D Q)}_I \\ &= Q^T D^m Q \\ &= Q^T \operatorname{diag}(\lambda_1^m, \lambda_2^m, \dots, \lambda_n^m) Q. \end{aligned}$$

Another example where this is useful: network analysis

Suppose we have the following:



Its connectivity matrix is: $A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$

i.e. A describes walks of size 1 (in other words, which nodes are connected to each other by 1 edge).

what is A^2 ?

$$A^2 = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}$$

This describes walks of size 2.
e.g.

to get from (2) to (2):

$$\begin{cases} (2) - (1) - (2) \\ (2) - (3) - (2) \end{cases}$$

$$\text{so } A^2_{(2,2)} = 2.$$

Knowing this, we would like to measure the importance of each node on the network, i.e. which nodes we come across the most for many different sizes of walks.

Strategy: Study walks of size $1, 2, \dots, \infty$:

$$f(A) = \sum_{p=0}^{\infty} C_p A^p \quad \left. \vphantom{\sum_{p=0}^{\infty}} \right\} \text{sum of weighted walks}$$

where each walk of size p is given a weight C_p .

Since we want to give more importance to shorter walks rather than long ones, C_p decreases as p increases. (and thus $f(A)$ converges).

$\Rightarrow$ Then we define the **subgraph centrality** of node i :

$[f(A)]_{ii}$
which can help us measure the importance of the node.

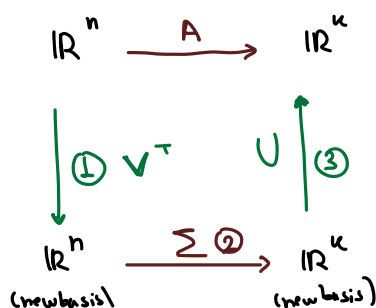
A more general case: non-square matrices

Since $A \in \mathbb{R}^{k \times n}$ with $k \neq n$, we no longer have a nice transformation

S invertible that can change my coordinates to a different basis.

Nonetheless, we can still find some transformation and do the following:

1. Transform old coord. to new coordinate system in $\mathbb{R}^n$



2. Multiply each component by a scalar and then "extend" / "project" them into $\mathbb{R}^k$

3. Transform back to old coord. system in $\mathbb{R}^k$

$\Rightarrow$ Singular value decomposition (SVD)

Given any matrix $A \in \mathbb{R}^{k \times n}$ it can be decomposed as:

$$A = U \Sigma V^T$$

rotation/reflection back $\nearrow$ scaling $\nwarrow$ rotation or reflection

where: • $V \in \mathbb{R}^{n \times n}$ is orthogonal (i.e. its columns are orthogonal)

• $\Sigma \in \mathbb{R}^{k \times n}$ is a "diagonal" matrix:

- only "diag." elements can be non zero

- $\Sigma_{jj} \geq 0$

- sorted decreasingly

- They are called singular values σ_i

Actually, singular values of A are the eigenvalues of $\sqrt{A^T A}$.

e.g. $\Sigma = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 4 \\ 0 & 0 & 0 & 0 \end{pmatrix} \in \mathbb{R}^{5 \times 4}$ or $\Sigma = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \end{pmatrix} \in \mathbb{R}^{3 \times 4}$

• $U \in \mathbb{R}^{k \times k}$ is orthogonal.

Remark: If $A \in \mathbb{R}^{k \times n}$ not square matrix, then $\text{rank}(A) \leq \min(k, n)$.

Then the number of singular values (including zeros) will be $\min(k, n)$

but the number of positive singular values will be $\text{rank}(A)$

This is why Σ contains some zero rows/columns.

E.g.

If $k > n$ (more rows) $\Rightarrow \underbrace{\min(k, n)}_{=: m} = n$ and $\Sigma = \begin{pmatrix} \overbrace{\sigma_1 \sigma_2 \dots \sigma_n}^n \\ \vdots \\ 0 \dots 0 \end{pmatrix} \left. \vphantom{\begin{pmatrix} \sigma_1 \\ \vdots \\ 0 \end{pmatrix}} \right\}^k$

$$= \begin{pmatrix} \Sigma_m \\ 0 \end{pmatrix}$$

If $k < n$ (more columns) $\Rightarrow m = \min(k, n) = k$ and $\Sigma = \begin{pmatrix} \sigma_1 & & & | & | & | \\ & \ddots & & | & | & | \\ & & \sigma_k & | & | & | \\ & & & 0 & 0 & \dots 0 \end{pmatrix}$

$$= \begin{pmatrix} \Sigma_m & 0 \end{pmatrix}$$

Then, we can drop these extra rows/columns and obtain a more compact form of the SVD:

$$\left. \begin{array}{l} \Sigma \in \mathbb{R}^{m \times m} \\ V^T \in \mathbb{R}^{m \times n} \\ U \in \mathbb{R}^{k \times m} \end{array} \right\} \begin{array}{l} \text{compact or reduced SVD} \\ \text{form:} \\ A = U \Sigma V^T \in \mathbb{R}^{k \times n} \end{array}$$

where $m = \min(k, n)$.

In Python: `np.linalg.svd(a, full_matrices=False)`.

• In general, Σ is stored as a vector.

How can we construct U, V and Σ ? Idea: simple decomposition for $A^T A \in \mathbb{R}^{n \times n}$.

Then $A^T A = Q^T D Q$ where D contains eigenvalues of $A^T A$. Now, if $A = U \Sigma V^T$,

then $A^T A = (U \Sigma V^T)^T (U \Sigma V^T) = V \Sigma U^T U \Sigma V^T = V \Sigma \Sigma V^T$.

Compare this to $Q^T D Q \Rightarrow Q = V$ and $D = \Sigma \Sigma$.

That is V is formed using the eigenvector of $A^T A$ and $\Sigma^2 = D$ implies Σ is made from the square root of the eigenvalues of $A^T A$, i.e., the singular values of A . Finally $U = A V \Sigma^{-1}$.

Applications to compressing

Given a matrix $A \in \mathbb{R}^{1020 \times 1080}$ corresponding to an image. Its SVD form:

$$A = U \Sigma V^T$$

$$(u_1 \dots u_n) \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_n \end{bmatrix}$$

$$= [\sigma_1 u_1 \quad \sigma_2 u_2 \quad \dots \quad \sigma_n u_n] \rightarrow U \Sigma V^T$$

$$= [\sigma_1 u_1 \quad \dots \quad \sigma_n u_n] \begin{bmatrix} v_1^T \\ v_2^T \\ \vdots \\ v_n^T \end{bmatrix}$$

$$= \underbrace{\sigma_1 u_1 v_1^T}_{\text{rank 1 matrix}} + \underbrace{\sigma_2 u_2 v_2^T}_{\text{rank 1 matrix}} + \dots + \underbrace{\sigma_n u_n v_n^T}_{\text{rank 1 matrix}}$$

why?

$$u_i v_i^T = \begin{bmatrix} v_{i1} u_i & v_{i2} u_i & \dots \end{bmatrix}$$

e.g. $\begin{pmatrix} 1 \\ 2 \end{pmatrix} (3 \ 4) = \begin{pmatrix} 3 \\ 6 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix}$

columns are linearly dependent.

Then, $A = \sum_{j=1}^{1080} \sigma_j u_j v_j^T$, i.e. A is the sum of 1080 rank 1 transformations.

But what if we use only the first ~~50~~ singular values and set the rest to 0? Then:

$$A = \sum_{j=1}^{50} \sigma_j u_j v_j^T$$

↳ Take the first 50 columns U
50 e-values Σ
50 rows V^T

i.e. $\Sigma \in \mathbb{R}^{50 \times 1}$ (stored as vector)

$$U \in \mathbb{R}^{1020 \times 50}, \quad V \in \mathbb{R}^{1080 \times 50}$$

The result should still be in $\mathbb{R}^{1020 \times 1080}$

How smaller is our new compressed system now?

$$1020 \times 50 + 50 + 1080 \times 50 = 3001 \times 50$$

vs

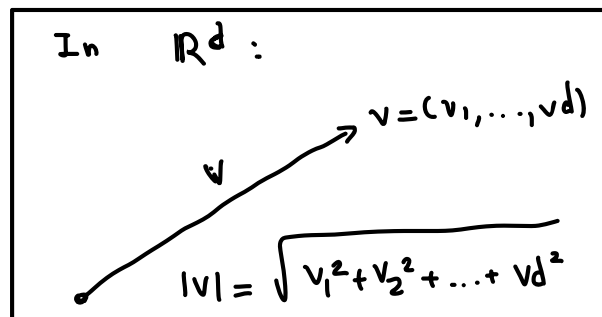
$$1020 \times 1080$$

$$\rightarrow \frac{3001 \times 50}{1020 \times 1080} \approx 0.072$$

That is, we are now only using about 7.2% of the original storage.

Full SVD will keep the quality of the image but might be too expensive.

Lecture 8



Properties

$$|v| = 0 \Leftrightarrow v = 0$$

$$|\lambda v| = \lambda |v|$$

$$|v + w| \leq |v| + |w| \quad (\text{T.I.})$$

define the distance between 2 points:

$$d(u, v) = |u - v|$$

Extract the most important properties of $| \cdot |$ and d so we can create new definitions of length and distance.

length $\Rightarrow$ norm $\| \cdot \| : \mathbb{R}^d \rightarrow [0, \infty)$

- $\|v\| = 0 \Leftrightarrow v = 0$
- $\|\lambda v\| = 0 \Leftrightarrow |\lambda| \|v\| = 0$
- $\|u + v\| \leq \|u\| + \|v\|$

Examples:

- $\|v\|_\infty := \max_j |v_j|$

- $\|v\|_1 := |v_1| + |v_2| + \dots + |v_d|$

- $\|v\|_2 := |v| \leftarrow \text{original length} \rightarrow \text{we call it } \underline{\text{Euclidean norm}}$

Utility: vectors can be understood as states, measurements of

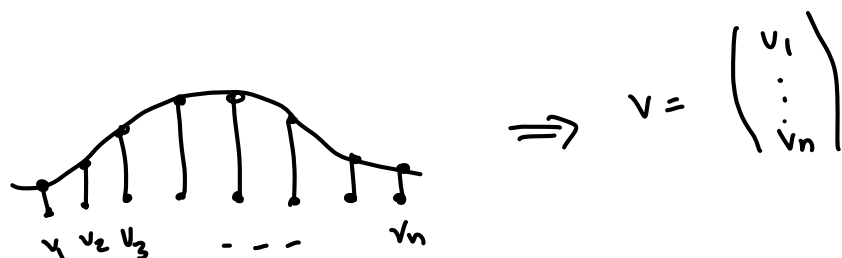
a physical system.

$\Rightarrow$ norm of a vector $\sim$ energy of said state.

e.g.

v : height of a water wave.

$\bullet$ $\|v\| = \sum_j |v_j|$ can be interpreted as the potential energy of the corresponding state.



Abstraction of distance: the metric

$d: X \times X \rightarrow [0, \infty)$ for any set X .
 $(x_1, x_2) \rightarrow d(x_1, x_2)$

Properties:

$\bullet d(x, x) = 0$

$\bullet d(x, y) = d(y, x)$ (symmetry)

$\bullet$ Triangle inequality: $d(x, y) \leq$

$d(x, z) + d(z, y)$



$\Rightarrow (X, d)$ metric space.

Good news, for normed spaces there's always a metric:

$$d(x, y) = \|x - y\|.$$

Another example: Genetic distance

Consider individual x , represented by

$$x = (x_1, \dots, x_n)$$

x_i gene at location i .

Each x_i is an integer $\Rightarrow$ This does not form a vector space.

(we haven't defined $\oplus$ or scalar $\lambda \cdot$)

$$x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}; \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

x and y are similar if most of

their genes coincide:

$$\begin{aligned} x_1 &= y_1 \\ x_2 &= y_2 \\ &\vdots \end{aligned}$$

$\Downarrow$
 $\longleftarrow$ However, we can still define a metric.

$d(x, y)$ will measure this similarity

$$d(x, y) = \# \{ i / x_i \neq y_i \}$$

e.g. $x = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{pmatrix}; y = \begin{pmatrix} 4 \\ 1 \\ 5 \\ 2 \\ 3 \end{pmatrix}$

$d(x, y) = 5.$

$\Rightarrow$ if $d \gg 0 \Rightarrow x, y$ are very different
 if $d \approx 0 \Rightarrow x, y$ are very close/similar.

"Hamming distance"

$\hookrightarrow$ can also be used to measure two different binary bits

Unfortunately not realistic.

$\hookrightarrow$ if $x = \begin{pmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \\ \begin{bmatrix} x_{n+1} \\ x_{n+2} \\ \vdots \end{bmatrix} \end{pmatrix} \quad y = \begin{pmatrix} \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \\ \begin{bmatrix} y_{n+1} \\ \vdots \end{bmatrix} \end{pmatrix}$

chromosome

Another possibility:
 Nei's standard distance.

$\left\{ \begin{array}{l} \text{If each chromosome of } x \text{ differs} \\ \text{by 1 gene with the respective} \\ \text{chromosome in } y, \text{ this distance} \\ \text{will say } x \text{ and } y \text{ are far apart} \\ \text{(not similar) for big samples.} \end{array} \right.$

locus: specific position on a chromosome
 where some genetic feature is observed.

e.g. chrom. 7, pos 12345678... = locus $\leftarrow$
location

At this locus
 different individuals
 may have different
 versions of genetic
material
variants

Assumption: for an individual / population u , at location k , there may be s_k possible variants:

$$u_k = \begin{pmatrix} u_{k,1} \\ u_{k,2} \\ \vdots \\ u_{k,s_k} \end{pmatrix} \Rightarrow u = (u_1 \ u_2 \ \dots \ u_k) \quad \begin{matrix} \uparrow \\ \text{total length.} \end{matrix}$$

big vector!

Define the distance as:
$$\rho(u, v) = -\ln \left(\frac{\sum_{k=1}^K \sum_{j=1}^{s_k} u_{k,j} v_{k,j}}{\sqrt{\sum_{k=1}^K \sum_{j=1}^{s_k} u_{k,j}^2 \sum_{k=1}^K \sum_{j=1}^{s_k} v_{k,j}^2}} \right)$$

Unpacking: If $u = (u_{11} \ u_{12} \ \dots \ u_{K,s_1}, u_{21}, u_{22}, \dots)$
 $v = (v_{11} \ v_{12} \ \dots \ v_{K,s_1}, v_{21}, v_{22}, \dots)$

$$\Rightarrow u \cdot v = \sum_{k=1}^K \left(\sum_{j=1}^{s_k} u_{k,j} v_{k,j} \right)$$

$$\begin{aligned} \Rightarrow \|u\|^2 &= u_{11}^2 + u_{12}^2 + \dots + u_{K,s_1}^2 + u_{21}^2 + u_{22}^2 + \dots \\ &= \sum_{k=1}^K \left(\sum_{j=1}^{s_k} u_{k,j}^2 \right) \end{aligned}$$

Same for v .

$$\Rightarrow \rho(u, v) = \ln \left(\frac{u \cdot v}{\|u\|^2 \|v\|^2} \right)$$

recall $\cos \theta = \frac{u \cdot v}{\|u\| \|v\|}$!

If profiles point in the same direction
 $\cos \theta \sim 1 \Rightarrow \ln(1) = 0 \Rightarrow$ they are similar

otherwise $\cos \theta$ gets smaller $\rightarrow$

and $\ln$ grows!

we measure this
inside our training
data.

e.g. Metrics in neural networks

• X set of all possible outputs { vectors, numbers
sequence of images
etc.

• choose a meaningful metric ρ on X .

$$\rho: X \times X \rightarrow [0, \infty)$$

so we can measure the true output x vs what we
predicted it to be, call it $\hat{x}$.

$\Rightarrow \rho(x, \hat{x})$ measures the error of the prediction.

In practice, we do this several times and then average these numbers.

Example (slide)

$\hookrightarrow$ just using $\|\cdot\|$ norm might not be enough if we don't
count scaling! $\Rightarrow$ To get a realistic norm, we should scale
with the length of the vector!!

Proper scaling in practice (notation: x true output, $\hat{x}$ prediction)

Since X is not a vector space (why?)

↳ X can contain vectors of dif. lengths, they can't be added.

Average distance

$$\Rightarrow d(x, \hat{x}) = \frac{1}{\text{length}(x)} \|x - \hat{x}\|_1$$

mean absolute error

we can use $\| \cdot \|_2$ as well but then $\rightarrow \sqrt{\text{length}(\hat{x})}$
root mean squared error.

what if $x = (x_1, \dots, x_n)$ is made out of digits?

$\Rightarrow$ distance can be measured by checking how many components coincide!

$$d_*(x, \hat{x}) = \frac{1}{\text{length}(x)} \cdot \sum_{j=1}^n \mathbb{1}_{x_j = \hat{x}_j}$$

Accuracy (if x_i is binary)

↓
binary accuracy

Problem: d_* is not a metric. Why?

↳ if $x = \hat{x} \Rightarrow d_*(x, \hat{x}) = 1$ (also similar $x, \hat{x} \Rightarrow d_* \approx 1$)
far apart $x, \hat{x} \Rightarrow d_* \approx 0$

$\Rightarrow$ better define $d(x, \hat{x}) = 1 - d_*(x, \hat{x})$

Hot encoding:

Example. 5 possible digits : 1, 2, 3, 4, 5.

We can assign one word. to each category:

Think more of them as categories.

digit	one-hot encoding
1	[1, 0, 0, 0, 0]
2	[0, 1, 0, 0, 0]
3	[0, 0, 1, 0, 0]
4	[0, 0, 0, 1, 0]
5	[0, 0, 0, 0, 1]

"exactly one entry"

Take $x = [\dots]$ s.t. each component of x is a category.

Example: suppose one component of $\hat{x}$ is [0.1 0.15 0.4 0.92 0.34]

Then we round up : $\text{round}(\hat{x}) = [0 \ 0 \ 0 \ 1 \ 0]$
maybe better to choose the maximum. "we predict category 4"

We can use $d_x(x, \hat{x}) \leftarrow \text{categorical accuracy.}$

Applying weights to different components.

Some components can be more important than others.

e.g. face detection: eyes.

we can weight some components $w_1, w_2, \dots, w_n$ s.t. $\sum w_i = 1$.

e.g. 4 categories $\left\{ \begin{array}{l} 1 \leftarrow w_1 \\ 2 \leftarrow w_2 \\ 3 \leftarrow w_3 \\ 4 \leftarrow w_4 \end{array} \right\} w = [0.6, 0.2, 0.1, 0.1]$

no no yes yes
 $x = [0, 0, 0, 1]$
 $\hat{x} = [1, 1, 0, 1]$

binary accuracy $w(x, \hat{x}) =$

$$\frac{1}{\text{length}(\hat{x})} \left(\sum_{x=\hat{x}} w_j \cdot 1 \right)$$

no need to divide because $\sum w_i$ already normalizes sum.

(without weights: $\frac{1}{4} (1+1) = 0.5$.)

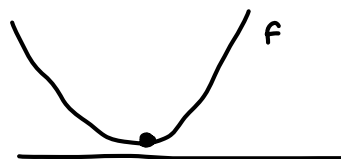
$$= (w_3 \cdot 1 + w_4 \cdot 1)$$

(e.g. say $w_i = \frac{1}{n}$)

$$= (0.1 \cdot 1 + 0.1 \cdot 1) = 0.2 !$$

low accuracy: we did not catch substantial components.

lecture 9 : Minimization and analysis



$$\Leftrightarrow Ax = b$$

Chief problem(s): minimization of multivariable functions $\xrightarrow{\text{Problem 1}}$ multiple dimensions possibly millions

$\xrightarrow{\text{Problem 2}}$ Does this exist?
Is it unique?

Goal for NN:
find parameters
s.t NN makes
good predictions

Specially in neural networks (NN)
many, many parameters

Good predictions
iff
Loss is minimal

we measure this
using a loss function
e.g. $L(\theta) = \frac{1}{N} \sum_{i=1}^N (\underbrace{\phi(x_i)}_{\text{neural network data}} - y_i)^2$ Mean squared error

Minimization problem
find parameter θ^* s.t
 $\theta^* = \underset{\theta}{\operatorname{argmin}} L(\theta)$

How can we find these parameters? θ^* .

* Millions of parameters
* Complicated $L(\theta)$ geometry and lack of nice properties $\rightarrow$ our only chance is to use numerical methods.

In general, we have to minimize $\phi: \mathbb{R}^n \rightarrow \mathbb{R}$ where:

$$\phi(s) = L(f(s, W_{in}), w_{out})$$

L loss function
 f NN
 s given input
 W_{in} parameters
 w_{out} given output
 L will depend on the structure of the network.
 L measures distance between output of NN: $f(s, W_{in})$ and the real output w_{out} .

Example: Suppose we have a sample of 10^4 images of size (480×460) each:

$$W_{in} = [W_1, \dots, W_{10^4}] \quad \text{s.t.} \quad W_i \in \mathbb{R}^{480 \times 460}$$

We will classify these images in 10 different groups.

$$\Rightarrow W_{out} = [W_1, \dots, W_{10^4}] \quad \text{s.t.} \quad W_i \in \mathbb{R}^{10} \quad (\text{e.g. one-hot encoding}).$$

Suppose our neural network does the following:

$$W_j \xrightarrow{A_1 x + b_1} A_1 W_j + b_1 \xrightarrow{A_2 x + b_2} A_2 (A_1 W_j + b_1) + b_2$$

Some tips

- ① These matrices are too dense. We need simpler ones

Parameters: $A_1 \in \mathbb{R}^{100 \times (480 \cdot 460)}$
 $A_2 \in \mathbb{R}^{10 \times 100}$

$\mathbb{R}^{480 \times 460}$
 Flatten this:
 $\in \mathbb{R}^{(480 \cdot 460)}$

$\mathbb{R}^{100}$
 Hidden layer with 100 neurons

$\mathbb{R}^{10}$
 $=: \hat{W}_j$
 predicted output

How?

Our loss function would have the shape

$$\phi(s) = \sum_{j=1}^{10^4} \|A_2(A_1 w_j) + b_1 - y_j\|^2$$

Reducing # of variables

↓
we avoid overfitting

Construct a more clever NN:

Use knowledge of data



e.g. With images, nearby pixels should be related

In our image example

$$W_j \in \mathbb{R}^{480 \cdot 640}$$

(if flattened). Then in

our hidden layer we

multiply $A w_j$:

$(480 \cdot 640) \cdot 100$ multiplications

On the other hand, with

$K \in \mathbb{R}^{3 \times 3}$ in our convolution

layer we multiply:

$K \cdot W_{\text{patch}} \Rightarrow 9$ multiplications

total # pixels

Per pixel

$\Rightarrow (480 \cdot 640) \cdot 9$ multiplications

Instead of flattening image into a vector W and transforming it with a dense matrix A , use the knowledge that nearby pixels are related



$$W = \begin{bmatrix} w_{11} & w_{12} & \dots \\ w_{21} & w_{22} & \dots \\ \vdots & \vdots & \ddots \\ w_{m1} & \dots & w_{mn} \end{bmatrix}$$

Remark: we could use several K 's (called filters)

multiply this "patch" of the image by a smaller transformation matrix $K = \begin{pmatrix} k_{11} & k_{12} \\ k_{21} & k_{22} \end{pmatrix}$

element-wise:

$$k_{11}w_{11} + k_{12}w_{12} + k_{21}w_{21} + k_{22}w_{22}$$

convolution layers!

Then, move to the next pixel and repeat process.